BUSINESS DATA MINING PROJECT · PRESENTATION SCRIPT

Architecture to Data mining

Script for sections 05–09 of the deck (slides 6 to 10 of 16). About 6 minutes, spoken in simple English, with pointing cues and answers to likely questions.

Run of show

Slide	Section	Title on the slide	Time
6	05	One pipeline from raw CSV to dashboard	1:00
7	06	New columns, and one fix that changed the answer	1:15
8	07	One nested document per shipment line	1:15
9	08	11 aggregation pipelines answer the questions	1:00
10	09	Four methods, one per kind of question	1:15
		Total	**5:45**

How to use this script

• Text in *[orange brackets]* is a cue (where to point or pause). Don't read it aloud.

• **Bold numbers** are the ones to stress. You don't need to memorise the rest word for word; speak naturally and keep the meaning.

• Each slide ends with a hand-over line that leads into the next slide, so the talk flows even if speakers change.

• The question boxes are short, safe answers. If you don't know something, say what the data shows and offer to check, rather than guessing.

• Be open about MongoDB: the pipelines are written and reviewed but have not been run on a live server yet.

Slide 6 of 16 · Section 05

Architecture: one pipeline from raw CSV to dashboard

Time: 1:00
Speaker: _____

On screen: A flow diagram: Kaggle CSV → Python ETL → MongoDB, Star-schema CSVs and Data mining → Power BI (navy box).

WHAT TO SAY

Before we go into each part, here is the whole system on one slide. *[point at the left box]*

We start with the raw Kaggle file: **10,324 rows** and **33 columns**. The first script is a Python ETL. ETL stands for extract, transform, load: it reads the file, cleans it and adds new columns.

From that one clean dataset we create three outputs. *[point at the middle column, top to bottom]* First, nested documents that we load into **MongoDB**, our database. Second, a **star schema**, one main table plus three lookup tables, which Power BI reads. Third, a flat table for our **data mining** script.

[point at the navy box] Everything ends up in the **Power BI dashboard**, including the data mining results such as the country segments and the association rules.

The key design idea is that every step is a numbered script, **01 to 05**. If the data changes, we re-run the scripts and the database, the models and the dashboard all update in a few minutes. Nothing is done by hand, so the whole project can be repeated.

Numbers to remember

Raw data: 10,324 rows × 33 columns (USAID SCMS, 2006–2015)

One ETL run → 3 outputs: MongoDB documents, star-schema CSVs, mining table

5 numbered scripts: 01 clean · 02 MongoDB · 03 mining · 04 build Power BI · 05 check Power BI

Dashboard: 5 pages, 67 visuals, 31 DAX measures

IF THE PROFESSOR ASKS

Q: Why send data to both MongoDB and CSV files?

A: MongoDB is our database for storing, validating and querying the data. Power BI works best with a star schema, so we give it one directly. Power BI can also connect live to MongoDB through its connector; that is an optional link.

Q: What does "ETL" mean?

A: Extract, Transform, Load: read the raw data, clean and reshape it, and save it where it will be used.

Hand-over line → "Let's look at what the cleaning step actually did."

Slide 7 of 16 · Section 06

Cleaning and features: new columns, and one fix that changed the answer

Time: 1:15
Speaker: _____

On screen: Left: a table of engineered features. Right: a dark box showing "US$ 107.9 M freight" crossed out and "US$ 68.8 M".

WHAT TO SAY

The cleaning script also created new columns. This is called **feature engineering**. *[point at the table]*

The most important one is **is_late**. We subtract the scheduled delivery date from the actual delivery date; if the result is positive, the shipment was late. This is the target our classification model predicts, and **11.5%** of line items are late.

We also grouped delays into four bands (early, on time, late 1 to 30 days, late more than 30 days), and calculated lead time from purchase order to delivery, freight per kilogram, freight as a share of value, total landed cost, and a region for each country.

[point at the dark box] Now the most important lesson of the project. When several products travel in one consignment, the dataset repeats the full freight cost on each of those lines: **2,445 lines** point to another row's freight. If you simply add up the freight column, you get **107.9 million dollars**. But that counts the same truck or plane many times.

When we count each consignment once, the real figure is **68.8 million**, about 39 million dollars less. *[pause]* Fixing this also changed our results. Our freight model's score dropped from 0.79 to an honest **0.64**, and one hypothesis, that fragile states pay far more for freight, turned out **not to be supported**. We see catching this as careful work, not a weakness.

Numbers to remember

is_late = delivered date − scheduled date > 0 → 11.5% of line items are late

11 engineered features (delay, status bands, lead time, freight per kg, freight %, landed cost, region …)

2,445 line items share a consignment's freight

Freight total: US$ 107.9 M (double-counted) → US$ 68.8 M (each consignment once)

Freight model R^2 after the fix: 0.64 (was 0.79 with duplicates)

IF THE PROFESSOR ASKS

Q: How did you notice the double-counting?

A: While building totals for the dashboard we saw that the shared rows carry exactly the same freight as the row they point to, so summing them repeats it. We added "allocated" columns where only the referenced line keeps the freight and the sharing lines carry zero.

Q: Why keep the copied value at all?

A: For per-consignment ratios like freight per kg, the copied freight together with the copied weight is correct. For totals we use the allocated column.

Q: Did you delete any rows?

A: No. All 10,324 rows were kept; unknown values became empty with a status flag, and every change is logged in data_quality_report.csv.

Hand-over line → "The cleaned data then goes into MongoDB."

Slide 8 of 16 · Section 07

MongoDB design: one nested document per shipment line

Time: 1:15
Speaker: _____

On screen: Left: a JSON document for shipment line 3. Right: a table of 5 collections, and a note on the validator, 6 indexes and $merge.

WHAT TO SAY

Why MongoDB? Each shipment line naturally has groups of details: where it went, how it travelled, who made it, what the product is, the dates and the costs. A document database lets us store each line as **one document with nested sections**, instead of 33 flat columns.

[point at the code] Here is a real example: shipment line 3 goes to Vietnam, directly from the vendor by air. It is Nevirapine made by Aurobindo, worth 6,200 dollars with about 4,500 dollars of freight, and it was delivered on time.

Our database is called **scms_lifeline**. *[point at the table]* The main collection, **shipments**, holds all **10,324** documents. We added a **JSON schema validator**: rules every document must follow. It must have a country, a scheduled and a delivered date, a value that is not negative, and a true-or-false late flag. Documents that break the rules are rejected.

We created **six indexes** so that filtering by country, date, transport mode and vendor is fast.

Finally, we built summary collections with the **$merge** stage: **vendors**, a scorecard for each of the 73 vendors; **countries**, a risk profile for each of the 43 countries; and **monthly KPIs** for trends. **ml_features** is a flat copy for the machine learning step.

Numbers to remember

Database scms_lifeline · main collection shipments = 10,324 documents

5 collections: shipments, vendors (73), countries (43), monthly_kpis (~110), ml_features (10,324)

$jsonSchema validator · 6 indexes · summaries built with $merge

Script: scripts/02_load_mongodb.py

IF THE PROFESSOR ASKS

Q: Why not just use SQL?

A: SQL would also work. We chose a document model because the data is hierarchical, and MongoDB's aggregation framework gives grouping, window functions and percentiles inside the database. For reporting we still use a star schema in Power BI, so each tool is used where it fits.

Q: What does $merge do?

A: It writes the result of an aggregation into another collection, like a saved summary table (a materialised view). Re-running the pipeline refreshes it.

Q: What is an index?

A: A lookup structure, like the index at the back of a book, that lets MongoDB find matching documents without scanning all of them.

Hand-over line → "So how do we ask the database questions?"

Slide 9 of 16 · Section 08

MongoDB queries: 11 aggregation pipelines answer the questions

Time: 1:00
Speaker: _____

On screen: A table of six pipelines (Q2, Q3, Q5, Q7, Q9, Q10) with their stages, and an orange "Status" pill.

WHAT TO SAY

In MongoDB you ask questions with **aggregation pipelines**. A pipeline works like an assembly line: the data passes through stages. **$match** filters, **$group** groups and counts, **$divide** calculates, **$sort** orders. We wrote **11 pipelines**, at least one for every research question.

[go down the table] **Q2** groups by transport mode and route to get the late rate of each combination, and keeps only groups with at least 30 line items so tiny groups don't mislead us. **Q3** uses **$percentile** to find the 90th-percentile delay per country: the bad-case delay, not just the average. **Q5** calculates freight per kilogram by mode and region.

Q7 builds the vendor Pareto with **$setWindowFields**, a window function that keeps a running total. **Q9** also uses a window, to compute a three-year moving average of the late rate. **Q10** compares the unit price of the same medicine across different vendors.

[point at the Status pill] To be transparent: these pipelines are written and reviewed, but we have **not yet run them on a live MongoDB server**. They need MongoDB 7.0 or later because they use $median, $percentile and window functions, and running them is our next step. The numbers we present come from our Python analysis and the Power BI dashboard.

PIPELINE Q2, IF YOU NEED TO SHOW IT

```
db.shipments.aggregate([
  { $group: { _id: { mode: "$fulfilment.mode", via: "$fulfilment.via" },
          n: { $sum: 1 },
          late_rate: { $avg: { $cond: ["$kpi.is_late", 1, 0] } } } },
  { $match: { n: { $gte: 30 } } },
  { $sort: { late_rate: -1 } }
])
```

Numbers to remember

11 pipelines in mongodb/aggregations.js (Q1–Q11)

Q2 keeps groups with n ≥ 30 · Q3 90th-percentile delay · Q7 and Q9 use window functions

Requires MongoDB 7.0+ ($median, $percentile, $setWindowFields)

Status: written and reviewed, not yet run on a live server

IF THE PROFESSOR ASKS

Q: Can you show one pipeline?

A: Yes, Q2 (printed above). It groups by mode and route, counts the line items, and averages a 1-or-0 late flag. The average of 1s and 0s is exactly the late rate. Then it drops small groups and sorts the worst first.

Q: Why haven't you run them yet?

A: MongoDB isn't installed on our machine yet. The cleaning, mining and dashboard did not depend on it, so we finished those first. Installing MongoDB and running the pipelines is the next task.

Hand-over line → "With the data clean and stored, we move to data mining."

Slide 10 of 16 · Section 09

Data mining: four methods, one per kind of question

Time: 1:15
Speaker: _____

On screen: Four cards: Classification (AUC 0.72), Regression (R^2 0.64), Clustering (3 segments), Association rules (Lift 2.3), and a line about fair testing.

WHAT TO SAY

We used four data mining techniques, one for each kind of question. *[go across the cards, left to right]*

Classification predicts a category, here a yes or no: will this shipment be late? Our best model, gradient boosting, scores an **AUC of 0.72**. In plain words, if you pick one late and one on-time shipment at random, the model rates the late one as riskier 72% of the time. 0.5 would be guessing.

Regression predicts a number, here the freight cost in dollars. Gradient boosting again, with an **R-squared of 0.64**; its average error is **63% lower** than a simple baseline.

Clustering groups similar things without being told the groups. K-means put 24 countries into **three segments** based on delay, transport mode and freight share.

Association rules find combinations that often happen together, like market-basket analysis in a supermarket. With the Apriori algorithm we found 66 rules that lead to late delivery. The strongest, trucked high-value consignments from regional warehouses, are late **2.3 times** as often as average.

[point at the bottom line] We were careful to test fairly. We trained the lateness model only on shipments scheduled **before 2013** and tested it on **2013 to 2015**, so it never sees the future. Mixing the years randomly gives a nicer-looking 0.89, but that is cheating, because the model learns from the future.

Numbers to remember

Classification: gradient boosting AUC 0.72 (goal 0.75; logistic regression 0.61; random split 0.89)

Train: 6,507 rows before 2013 (9.3% late) · Test: 3,817 rows 2013–15 (15.3% late)

Regression: R^2 0.64 · MAE US$ 4,006 vs US$ 10,883 baseline (63% lower) · 5,963 unique consignments

Clustering: K-means, 24 countries, k = 3, silhouette 0.33

Association rules: Apriori, 66 rules → LATE · top rule 26.2% late vs 11.5% average → lift 2.28

IF THE PROFESSOR ASKS

Q: What is gradient boosting?

A: Many small decision trees built one after another; each new tree corrects the mistakes of the ones before it. It handles mixed categories and numbers and non-linear effects well.

Q: Why AUC and not accuracy?

A: Only 11.5% of shipments are late. A model that always says "on time" is 88.5% accurate but useless. AUC measures how well the model ranks risky shipments, regardless of that imbalance.

Q: What does lift 2.28 mean?

A: These shipments are late 26.2% of the time, against an average of 11.5%. 26.2 ÷ 11.5 = 2.28, so they are late 2.28 times as often.

Hand-over line → "Let's look at the prediction results in more detail." (slide 11)

One-page cheat sheet

Keep this in front of you while presenting slides 6–10.

Topic	Number
Dataset	10,324 rows × 33 columns · 43 countries · US$ 1.63 bn · 2006–2015
Late line items	11.5% overall · From RDC 17.2% vs Direct Drop 5.3%
Shared-consignment rows	2,445
Freight total	US$ 68.8 M (not 107.9 M) · 4.5% of commodity value
MongoDB	scms_lifeline · 10,324 docs · 5 collections · 6 indexes · validator · 11 pipelines
MongoDB status	Written and reviewed; live-server run is the next step (needs 7.0+)
Classifier	AUC 0.72 (goal 0.75) · time split: train before 2013, test 2013–15
Freight model	R^2 0.64 · MAE US$ 4,006 vs 10,883 baseline · error 63% lower
Clustering	k = 3 segments · silhouette 0.33 · 24 countries
Association rules	66 rules → LATE · top rule lift 2.28 (26.2% vs 11.5%)
Scripts	01 clean · 02 MongoDB · 03 mining · 04 Power BI · 05 check

Plain-English definitions

ETL: read, clean and save the data. **Feature**: a column the model learns from.
Document database: stores each record as a flexible JSON-like document.
Aggregation pipeline: a chain of steps (filter, group, calculate) run inside MongoDB.
AUC: ranking quality, 0.5 = guessing, 1 = perfect. **R^2**: share of variation explained.
MAE: average error in dollars. **Silhouette**: how well separated clusters are.
Support / confidence / lift: how common / how often late / how much more than average.